HACKHUB

Email Verification (OTP) Interview Q&A

Code-grounded preparation for fresher and junior developer interviews.

Signup and login, OTP generation, HMAC storage, Postgres atomicity, expiry and attempts, rate limits, SMTP delivery, bcrypt, and the scenarios recruiters probe.

45 **QUESTIONS**
Built from the actual implementation, including honest gaps.

CONTENTS

1. Project Walkthrough	Q1-Q6
2. Email Verification Basics	Q7-Q11
3. OTP Generation	Q12-Q17
4. OTP Storage	Q18-Q23
5. OTP Lifecycle and Abuse Prevention	Q24-Q30
6. Password Hashing	Q31-Q35
7. Email Delivery and Account Security	Q36-Q41
8. Scenarios	Q42-Q45
Gaps and How to Answer Honestly	11 gaps
Quick-Fire Concepts	14 rows
30-Second Project Pitch	1 pitch

Generated on 2026-11-03. The answers describe the current working tree, including limitations.

1. Project Walkthrough Q1-Q6

These questions trace the real request path from POST /api/auth/signup through the Postgres RPCs and the VerifyEmail page. The goal is to explain the design and its trade-offs, not to memorize a script.

Q1. Walk me through what happens from signup to a verified email.

Answer: Signup is a POST to /api/auth/signup: I validate the fields, create the user with bcryptjs and email_verified=false, generate a six-digit OTP, HMAC-SHA256 it, store the hash through the issue_email_verification_otp RPC, and send the code with Nodemailer. Signup itself does not create a session; the browser goes to /verify-email, posts email plus OTP to /api/auth/verify-otp, and verify_email_otp checks expiry and attempts, consumes the code, and flips email_verified. The important concept is that the database is the source of truth, not the browser. In my project I kept every state transition server-side because a client-side "verified" flag can be tampered with.

Q2. Why did you choose OTP instead of a magic link?

Answer: An OTP is a short-lived secret the user copies from an email; a magic link is a one-click URL containing a bearer token. A link is smoother UX, but it puts a token in browser history, referrer headers, and email scanners, while an OTP keeps verification inside one page and is easy to resend. The trade-off is that users can be tricked into sharing an OTP, and a link can be scanned automatically. In my project I chose OTP because VerifyEmail.jsx already provides a six-box input, and the code is bound to the user id in the HMAC instead of being a standalone URL token.

Q3. What was the hardest problem you had to solve?

Answer: The hardest part is making resend and verify safe when two requests arrive together: without locking, two active codes could exist or one code could be consumed twice. I solved that with a Postgres RPC that locks the users row with SELECT FOR UPDATE, consumes old codes, and a unique partial index that allows only one unconsumed OTP per user. The trade-off is more PL/pgSQL logic and database coupling; in my project that is acceptable because Supabase Postgres is already the system of record.

Q4. What would you improve next?

Answer: If I had more time I would add password reset, enforce verification server-side on every sensitive route, move rate limiting to Redis, and add key rotation for the OTP secret. Those changes reduce account-recovery dead ends, bypass risk, and single-process limits. The trade-off is more infrastructure and migration work. In my project the verified-submission gate already calls requireVerifiedEmail, but the next step is to share that gate across other account actions.

Q5. How did you test it?

Answer: I tested the pure OTP logic with node:test in backend/tests/otpService.test.js: five thousand generated codes must be six numeric digits, HMAC must be deterministic and user-bound, issue must return an expiry ten minutes ahead and never return the hash, cooldown mapping must work, and revoke must delete. All six tests pass. The gap is that these are unit tests with a fake repository, so they do not prove the SQL functions, route behavior, email delivery, or concurrent requests; in my project the database RPCs still need integration tests.

Q6. Where exactly is email verification enforced?

Answer: Email verification should gate the action that creates real-world trust or costs money; gating every page hurts onboarding. My code checks requireVerifiedEmail only on POST /api/hackathons, while login still issues a session for an unverified user and the frontend redirects them to /verify-email. That is a deliberate trade-off: browsing remains easy, but organizer submissions are protected server-side. In my project the database check in requireVerifiedEmail is what matters because the frontend redirect alone is not a security boundary.

2. Email Verification Basics Q7-Q11

These questions separate mailbox proof from authentication and explain why the server must own the decision.

Q7. Why verify an email address at all?

Answer: Verifying an email proves the account holder can receive mail at that address, which reduces fake accounts, prevents typos from locking users out, and gives you a trusted recovery channel. It is not a full identity check. The trade-off is extra signup friction and dependence on email deliverability. In my project I use it to gate organizer submissions and to exclude unverified users from the daily digest query in listUsersForDigest.

Q8. What is the difference between verification, identity, and login?

Answer: Verification answers whether this person controls this mailbox, authentication answers which session this is, and identity is the stored user record. They are separate because a user can be logged in but unverified, or verified but logged out. In my project users.email_verified is the verification claim, the signed JWT cookie is authentication, and login deliberately allows an unverified session so the frontend can route the user to VerifyEmail.jsx.

Q9. Why must OTP validation happen server-side?

Answer: Server-side validation means the backend re-checks the OTP, expiry, attempt count, and user state even if the browser says the form is valid. Client checks are only UX because DevTools or a direct HTTP client can bypass them. In my project VerifyEmail.jsx checks six digits for convenience, but /api/auth/verify-otp revalidates with express-validator and verify_email_otp rechecks expiry, attempts, and the hash in Postgres.

Q10. What does email verification prove, and what does it not prove?

Answer: Verification proves control of the mailbox at that moment; it does not prove the person's real name, age, or that they will keep the mailbox. A recycled or compromised mailbox can later receive a reset link, so verification has a time boundary. In my project I store email_verified_at and bind the OTP HMAC to the user id, which makes the proof auditable and prevents a hash from being replayed on another account.

Q11. Why not trust the emailVerified value sent by the client?

Answer: The client's emailVerified value is display state and can be stale or modified, so it must never authorize an action. The server has to read the current user row and decide. In my project requireVerifiedEmail calls findById(req.userId), checks user.emailVerified, and returns 403 EMAIL_NOT_VERIFIED; the JWT only carries the user id in sub, not a trusted verification flag.

3. OTP Generation Q12-Q17

The code is short by design, so randomness, formatting, and the expiry window carry a large part of the security burden.

Q12. How do you generate the OTP?

Answer: OTP generation must use a cryptographically secure random source; a CSPRNG prevents an attacker from predicting the next code from previous ones. I use crypto.randomInt(0, 10**6), which returns a uniform integer from 0 to 999999, then padStart(6, '0') keeps leading zeros. The trade-off is only one million possible codes, so the short TTL, five-attempt cap, and rate limits carry much of the security weight. In my project generateOtp is intentionally simple because the layered controls, not the code length alone, provide the protection.

Q13. How much entropy does a six-digit OTP have?

Answer: A six-digit OTP has log2(1,000,000), about 19.9 bits of entropy, which is small for offline guessing but workable for a short online flow. An attacker has at most five guesses per code, and the request and verify limiters slow repeated attempts. In my project I accepted that trade-off for usability; if the threat model were higher, I would use eight digits or a longer random token.

Q14. Why not use TOTP or HOTP instead?

Answer: TOTP is time-based and HOTP is counter-based; both are usually paired with an authenticator app and a shared secret enrolled earlier. They are excellent second factors, but they require an enrollment and recovery flow and do not by themselves prove mailbox ownership. The trade-off is more setup friction and a shared-secret enrollment problem, and both approaches can still be phished. In my project I use email OTP because the goal is signup mailbox verification, not login two-factor authentication, and there is no authenticator enrollment.

Q15. Why keep OTPs as strings instead of numbers?

Answer: Leading zeros matter because 000123 is a valid six-digit code, while storing it as a number would turn it into 123 and break verification. I keep OTPs as strings from generation through validation, and the six-digit regex enforces exactly six digits. The risk is accidental numeric coercion somewhere in the stack. In my project isSixDigitOtp and the express-validator matches rule both work on strings, so the format survives the HTTP and database boundaries.

Q16. What is the OTP TTL and where is it enforced?

Answer: TTL is the maximum age of a code; once expires_at is in the past, the code must be rejected even if it was never used. I set OTP_TTL_MS to 10 * 60 * 1000, compute expiresAt in otpServiceCore, and verify_email_otp compares against now() and consumes the row when it is expired. The risk is clock skew or a template that hardcodes the TTL. In my project the database is authoritative for expiry, although the email text currently hardcodes "10 minutes".

Q17. Why is Math.random unacceptable here?

Answer: Math.random is not cryptographically secure; its output can be predicted from enough samples and is unsuitable for secrets. crypto.randomInt uses the operating system CSPRNG and avoids modulo bias. In my project generateOtp uses crypto.randomInt and a 5000-iteration test checks the six-digit string shape, though that test does not statistically prove randomness.

4. OTP Storage Q18-Q23

The database should never contain a usable code. These questions cover the HMAC choice, the secret, comparison, and rotation.

Q18. Do you store OTPs in plaintext?

Answer: Storing the OTP itself would turn any database read into account takeover, and reversible encryption would still leave a decryption key on the server, so I store a one-way keyed digest instead. hashOtp computes HMAC-SHA256(secret, userId + ':' + otp) and stores the 64-character hex string in email_verification_otps.otp_hash; the plaintext exists only in memory and the email. In my project the user-id binding means one user's hash cannot be replayed against another account.

Q19. Why is plain SHA-256 weak for a six-digit OTP?

Answer: A plain SHA-256 hash of a six-digit OTP is weak because the entire input space is only one million values, so an attacker with a database dump can test every possibility almost instantly. A salt prevents precomputed tables but does not make a fast hash slow enough. In my project I use HMAC-SHA256 with a server secret and bind the user id, which raises the bar if only the database leaks, though it is still fast if the HMAC secret also leaks.

Q20. Why HMAC instead of bcrypt or Argon2 for the OTP?

Answer: HMAC is a keyed MAC: it lets me verify a short code without storing it, and it is deterministic, so a database lookup by user id is straightforward. bcrypt or Argon2 are deliberately slow, but they are better suited to passwords because they need a per-row salt and comparison rather than an index-friendly digest. In my project I chose HMAC-SHA256 because OTPs live for ten minutes and the real protection comes from secrecy of the key plus rate limits; the trade-off is fast offline guessing if that key is exposed.

Q21. How is the HMAC secret managed?

Answer: Secret management means keeping the HMAC key out of source control, injecting it at runtime, and separating it from other keys so rotation and blast radius are controlled. resolveOtpSecret reads OTP_HASH_SECRET and falls back to JWT_SECRET only if the dedicated value is missing. The risk is that the fallback couples OTP invalidation to session-token rotation and weakens key separation. In my project .env is gitignored and .env.example contains only placeholders, but the working environment does not set OTP_HASH_SECRET, so the fallback is currently used.

Q22. Do you use a timing-safe comparison?

Answer: A timing-safe comparison avoids leaking how many leading bytes matched; crypto.timingSafeEqual is the usual Node tool for fixed-length secrets. My verify path does not call it: the HMAC strings are compared inside verify_email_otp with a SQL inequality check. In my project the verify limiter and five-attempt database cap reduce remote timing risk, but I should not claim a constant-time compare because the code does not implement one.

Q23. How would you rotate the OTP secret?

Answer: Key rotation means replacing the HMAC secret without invalidating every active code at once, usually with a key id or a dual-secret verification window. My schema has no key version column and resolveOtpSecret uses a single secret, so changing it invalidates all outstanding OTPs, and if it falls back to JWT_SECRET, changing the JWT secret does the same. In my project that is acceptable for a small app but is a real operational limitation I would fix with otp_key_version and a grace period.

5. OTP Lifecycle and Abuse Prevention Q24-Q30

These questions cover attempts, cooldowns, rate limiting, replay, and the atomic database functions that make the lifecycle safe under concurrency.

Q24. How many OTP attempts are allowed?

Answer: The attempt limit caps how many wrong guesses one OTP allows; without it a six-digit code is brute-forceable over time. OTP_MAX_ATTEMPTS is exported as 5, and verify_email_otp increments attempts on each mismatch, consumes the row at five, and returns attempts_exceeded. The risk is duplicated policy: the SQL function hardcodes 5 and the check constraint also uses 5. In my project the database is the enforcing layer, but I would generate the SQL limit from one configuration source.

Q25. How does the resend cooldown work?

Answer: Resend cooldown prevents a user or attacker from generating a new code immediately and resetting the attempt counter. My issue_email_verification_otp RPC looks at max(created_at), compares it with p_resend_after_seconds (60), and returns status cooldown with retry_after_seconds. The UI disables the resend button and counts down, but the database is authoritative. In my project the 60-second cooldown applies even after a code is consumed, which is intentional to stop rapid resend loops.

Q26. What rate limits did you add?

Answer: Rate limiting caps request volume per key and window; it is a blunt but important layer against brute force and email bombing. I use express-rate-limit with authLimiter at 20 requests per 15 minutes per IP, otpRequestLimiter at 6 per 15 minutes per email plus IP, and otpVerifyLimiter at 20 per 15 minutes per email plus IP. The trade-off is that the default store is in-memory, the server does not set trust proxy, and multiple instances do not share counters. In my project the database cooldown and five-attempt cap still apply, but production would need a Redis-backed store and correct proxy configuration.

Q27. Is OTP verification atomic?

Answer: Atomic verify-and-consume means checking the code, counting the failure, marking it consumed, and setting the user verified as one transaction, so a crash cannot leave half the state changed. verify_email_otp locks the users row and the active OTP row with FOR UPDATE, then performs all updates inside the same function call. The trade-off is that the logic lives in PL/pgSQL and is harder to unit test than plain JavaScript. In my project I chose the RPC because it closes race conditions that separate Supabase queries would leave open.

Q28. How do you ensure only the latest OTP is valid?

Answer: Latest-OTP-only means a resend invalidates the previous code so users cannot accumulate valid codes. My issue function marks every unconsumed row for that user as consumed before inserting the new one, and idx_email_verification_otps_one_active is a unique partial index on user_id where consumed_at is null. The risk is that a bug in the function would hit the unique index, which is actually a useful safety failure. In my project the unique index and the user-row lock work together to keep exactly one active code.

Q29. How do you prevent OTP replay?

Answer: Replay prevention means a code that succeeded once cannot be used again, even by someone who captured it. verify_email_otp selects only rows with consumed_at is null, and on success it sets consumed_at = now() before returning verified. A second request then gets not_found or invalid rather than a second success. In my project the consume happens in the same transaction as the user update, so there is no window where the code is both usable and verified.

Q30. Do you lock the account after repeated failures?

Answer: Account lockout is a stronger control that temporarily blocks the account after repeated failures; OTP lockout is narrower because it blocks only the current code. My code has no permanent account lockout: after five bad attempts the OTP is consumed, but a user can request a new code after the cooldown, and password login is limited only by the IP-based authLimiter. The trade-off is less protection against a patient attacker but fewer support problems; in my project the short TTL, cooldown, and rate limits are the intended compensating controls.

6. Password Hashing Q31-Q35

Email verification is separate from password storage. These questions cover the bcrypt implementation and the missing reset flow.

Q31. Why hash passwords?

Answer: Password hashing is a one-way, salted, deliberately slow transformation so a database leak does not reveal passwords and identical passwords do not produce identical hashes. bcrypt embeds the salt and cost in the hash string and is designed to resist offline cracking. The trade-off is CPU cost and the 72-byte bcrypt input limit. In my project createUser calls bcrypt.hash(password, 10) with bcryptjs, and comparePassword uses bcrypt.compare against the raw row only during login.

Q32. Why bcryptjs instead of Argon2?

Answer: bcrypt is mature and widely supported; Argon2 is memory-hard and generally the modern first choice for new systems. bcryptjs is pure JavaScript, so cost 10 is slower than native bcrypt and there is no memory-hardness. In my project I kept bcryptjs because it was already a dependency and the project is small; for a high-scale system I would benchmark native bcrypt or migrate to Argon2id.

Q33. What do the salt and cost factor do?

Answer: A salt makes each hash unique so precomputed tables and cross-user comparisons fail; the cost factor makes each guess expensive. bcrypt generates a random salt internally, and cost 10 means 2^10 key-expansion rounds. The risk is choosing a cost that is too low for current hardware or too high for the server. In my project bcrypt.hash(password, 10) uses the library's automatic salt, and I would revisit the cost with production benchmarks.

Q34. Is there a password reset flow?

Answer: A password reset flow normally issues a single-use, high-entropy token, stores only its hash, expires it quickly, and invalidates sessions after the change. My codebase has no forgot-password route, reset token table, or reset UI; only login, signup, verify-otp, resend-otp, me, and logout exist in auth.js. The risk is that a user who forgets a password, especially an unverified one, has no self-service recovery and needs manual support. In my project this is an honest missing feature, not something I would claim in an interview.

Q35. What is your password policy?

Answer: Password policy controls what users can choose; hashing controls what happens after storage, and transport controls how the password reaches the server. My validation only enforces a minimum of eight characters, with no maximum, breach check, or strength meter, and bcrypt truncates beyond 72 bytes. The risk is weak passwords and silent truncation for very long passphrases. In my project I would add a maximum length and a breached-password check, but I should not claim those controls exist today.

7. Email Delivery and Account Security Q36-Q41

Delivery, enumeration, logs, and the difference between an OTP and second-factor authentication are common recruiter follow-ups.

Q36. How is the OTP email sent?

Answer: Email delivery is the step that turns a stored OTP into something the user can actually receive; an API 201 only means the request was accepted, not that the message reached the inbox. I use Nodemailer with SMTP_HOST, SMTP_USER, and SMTP_PASS, port 587 by default and secure only on 465, and send both text and HTML bodies. The risk is spam filtering or a silent bounce after sendMail resolves. In my project sendOtpEmail awaits the SMTP response and the API returns a delivery-failed error on a throw, but there is no bounce webhook or inbox confirmation.

Q37. Are OTPs ever written to logs?

Answer: OTP logging is dangerous because logs are often less protected than the database and may be exported to third-party systems. My code does not log the OTP itself: the value is passed to formatOtpText and formatOtpHtml, and morgan logs request metadata such as method, URL, status, and timing, not the JSON body. The residual risk is that the SMTP provider or the recipient's mail system stores the message. In my project the OTP is never placed in a URL or an application log, which is the part I can control.

Q38. Does your implementation leak whether an account exists?

Answer: Account enumeration happens when responses differ enough to tell whether an email is registered. My code intentionally distinguishes states: signup returns EMAIL_ALREADY_REGISTERED versus ACCOUNT_PENDING_VERIFICATION, resend returns 404 ACCOUNT_NOT_FOUND, login returns ACCOUNT_NOT_FOUND versus INVALID_PASSWORD, and verify returns EMAIL_ALREADY_VERIFIED. The risk is that an attacker can map valid emails and target phishing or credential stuffing. In my project those messages were chosen for clearer UX, but the security trade-off is real; I would use uniform responses and send the real outcome only by email.

Q39. Is email OTP a second factor here?

Answer: Email OTP is a possession factor when it is used alongside a password; it is not automatically two-factor authentication just because a code is emailed. My flow uses the OTP to prove mailbox ownership during signup, not as a second login factor: login still checks the password and then issues a session, and unverified users are redirected to verification. The risk is that a compromised mailbox can intercept the code. In my project the OTP is a verification gate, not a replacement for the password.

Q40. How secure is the SMTP configuration?

Answer: SMTP security depends on TLS, certificate validation, and correct port settings; port 587 normally uses STARTTLS, while port 465 uses implicit TLS. My getMailer sets secure: port === 465 and passes the credentials from environment variables, but it does not explicitly set requireTLS, verify the connection at startup, or pin a certificate. The risk is misconfiguration or a downgrade depending on the provider. In my project I rely on Nodemailer and the SMTP provider defaults, and I validate only that the three required variables are present.

Q41. What happens if email delivery fails?

Answer: Delivery failure handling should avoid leaving an account or OTP in a state the user cannot recover from. In deliverVerificationCode I revoke the issued OTP if sendOtpEmail throws, and the signup route deletes the brand-new unverified user as a best-effort rollback. The risk is that the cleanup itself fails or the SMTP timeout is ambiguous because the message may still arrive. In my project these are compensating actions, not a distributed transaction, so an outbox or retry job would be the production fix.

8. Scenarios Q42-Q45

These are the scenario questions that test whether you understand the failure modes, not just the happy path.

Q42. A user enters the correct OTP but it is rejected. Why?

Answer: If a correct OTP is rejected, the usual causes are an expired row, a newer code having consumed the old one, a changed HMAC secret, a format mismatch, or the attempt cap already being reached. My verify path checks not_found, expires_at, attempts, and the hash in that order, and the frontend clears the digits after an error. The risk is a confusing support case if the user pasted spaces or requested a resend. In my project I would add structured audit logging around those statuses without logging the OTP itself.

Q43. What happens if two verify requests arrive at the same time?

Answer: Concurrent requests are the classic race: two verifies could both read the same row, or two issues could create two active codes. My RPCs lock the users row first, then the active OTP row, and the unique partial index is a second guard; the first request wins and the second sees consumed or not_found. The trade-off is lock contention for one user, which is irrelevant at this scale. In my project I chose database locks over application-level mutexes because multiple server instances would not share a mutex.

Q44. What if the HMAC secret leaks?

Answer: If the HMAC secret leaks, an attacker can brute-force the six-digit input space for each stored hash because HMAC-SHA256 is fast; the short TTL limits the window but does not make the hash slow. The fix is to rotate the secret, add a key version and dual-verify window, and consider a slower KDF if the threat model justifies it. In my project resolveOtpSecret uses a single secret with no versioning, so rotation currently invalidates every outstanding code and there is no graceful overlap.

Q45. What if the OTP table leaks but the HMAC secret does not?

Answer: If the OTP table leaks without the secret, the hashes are not directly usable, but email addresses, timestamps, attempts, and account state still leak metadata. The HMAC user binding also means an attacker cannot move a hash to another account. In my project the service role key stays server-side and RLS is enabled on email_verification_otps, but a database dump would still expose the hash material; I would treat secret compromise separately from database compromise.

Gaps and How to Answer Honestly

11 gaps

These are real limitations in the current code. Do not hide them: name the weakness, explain the risk, describe the fix, and say it confidently.

Gap 1. Unverified users can still log in.

Weakness: The login route calls sendSession for any matching email and password, regardless of email_verified. The frontend checks loggedInUser.emailVerified === false and redirects to /verify-email, but the backend does not reject the session.

Risk: An unverified account can call requireAuth-only endpoints such as profile updates, saving hackathons, and recommendations. Only POST /api/hackathons is protected by requireVerifiedEmail.

How I would fix it: Check email_verified in the login route, or add requireVerifiedEmail to every sensitive route; keep a narrow allowlist for verification and logout.

How to say it: *"I allow unverified login so users can reach the verification screen, but I do not treat that as a security boundary. The submission endpoint rechecks email_verified in the database, and my next step is to move that check into shared middleware."*

Gap 2. The API reveals whether an account exists.

Weakness: Signup distinguishes verified from pending accounts, resend returns 404 for an unknown email, login distinguishes a missing account from a wrong password, and verify returns EMAIL_ALREADY_VERIFIED.

Risk: An attacker can enumerate valid accounts and use that list for phishing, credential stuffing, or targeted OTP requests.

How I would fix it: Return uniform public responses, send state-specific detail only by email, rate limit by normalized email and IP, and add bot detection where abuse appears.

How to say it: *"My current messages favor UX over anti-enumeration. I know that leaks account existence, and I would move the detail into email while keeping the public API response generic."*

Gap 3. Rate limiting is per process and proxy-sensitive.

Weakness: express-rate-limit uses the default in-memory store, server.js does not set trust proxy, OTP keys use email plus req.ip, and counters are not shared across instances.

Risk: A multi-instance or proxied deployment can bypass or mis-key the limits, and an attacker can vary email or IP to spread requests.

How I would fix it: Use a Redis-backed store, configure trust proxy correctly, combine IP and normalized email keys, and add a per-account global limit.

How to say it: *"The database cooldown and five-attempt cap are the durable controls. The HTTP limiter is a best-effort layer, and I would move it to Redis and configure trust proxy for production."*

Gap 4. There is no explicit timing-safe OTP comparison.

Weakness: verify_email_otp compares otp_hash with a SQL inequality check. There is no crypto.timingSafeEqual call in the Node code.

Risk: A timing side channel is theoretically possible, although remote exploitation is difficult because of the verify limiter, the five-attempt cap, and the comparison happening inside Postgres.

How I would fix it: Fetch the active hash and compare it with crypto.timingSafeEqual, or use a database comparison specifically designed for constant-time behavior; keep the rate limits.

How to say it: *"I do not claim a timing-safe compare. The hash comparison is in SQL, and the rate limiter and attempt cap are the practical mitigations; I would add an explicit constant-time compare if this were audited."*

Gap 5. The OTP secret currently falls back to JWT_SECRET.

Weakness: resolveOtpSecret prefers OTP_HASH_SECRET but falls back to JWT_SECRET. The working .env does not set OTP_HASH_SECRET, there is no minimum-length validation, and there is no key version or rotation window.

Risk: Key reuse couples OTP invalidation to session-secret rotation; if the secret leaks, an attacker can brute-force the one-million-value OTP space. Rotating the secret invalidates every outstanding code.

How I would fix it: Require a dedicated, long random OTP_HASH_SECRET at startup, validate its length, add a key version with a dual-secret grace period, and rotate it on a schedule.

How to say it: *"The code supports a dedicated OTP secret, and the example env recommends it, but my local env currently uses the JWT fallback. I would make the dedicated secret mandatory and add versioned rotation."*

Gap 6. Password reset is not implemented.

Weakness: There is no forgot-password route, reset-token table, or reset UI. auth.js only has login, signup, verify-otp, resend-otp, me, and logout.

Risk: A user who forgets a password, especially an unverified user, has no self-service recovery and needs manual support.

How I would fix it: Add a single-use, high-entropy reset token stored as a hash with a short TTL, invalidate sessions after the password change, and notify the user.

How to say it: *"Password reset is not implemented yet. I would not pretend it is; the design would use a single-use hashed token with a short TTL and session invalidation."*

Gap 7. Account creation and email delivery are not one transaction.

Weakness: The user insert, OTP insert, SMTP send, and cleanup are separate operations. Rollback is best effort: revoke the OTP and delete the unverified user.

Risk: An orphan user or active OTP can remain if cleanup fails, and an SMTP timeout is ambiguous because the message may still be delivered.

How I would fix it: Use an outbox or queue for email, make sends idempotent with retries, and avoid deleting an account after an ambiguous send.

How to say it: *"The database RPCs are atomic, but the email side is not. I handle failure with revoke plus delete as a best effort; a production design would use an outbox and retries."*

Gap 8. Tests do not cover the database, routes, email, or concurrency.

Weakness: There is one test file, backend/tests/otpService.test.js, with a fake repository. There are no SQL-function, route, SMTP, frontend, or concurrency tests.

Risk: A regression in attempts, cooldown, RLS, permissions, or race behavior can pass the current unit tests.

How I would fix it: Add integration tests against a disposable Postgres or Supabase instance, mock SMTP, and run concurrent issue and verify requests.

How to say it: *"My six unit tests cover the pure OTP logic and pass, but they do not prove the database functions. I would add integration and concurrency tests before calling this production-ready."*

Gap 9. OTP policy values are duplicated and partly hardcoded.

Weakness: OTP_MAX_ATTEMPTS is exported but unused in the service, the SQL function hardcodes 5, the email text hardcodes ten minutes, and the frontend hardcodes OTP_LENGTH 6.

Risk: Changing one constant can leave the database, email, or UI out of sync, and users can be told the wrong expiry.

How I would fix it: Pass configuration into SQL, template the TTL into the email, and centralize shared constants so there is one source of truth.

How to say it: *"The values are consistent today, but they are duplicated. I would make the database the single source for the attempt cap and template the TTL into the email."*

Gap 10. There is no CAPTCHA or global email-bombing protection.

Weakness: Rate limits exist, but there is no CAPTCHA, proof of work, or per-account global send limit. The resend endpoint is unauthenticated.

Risk: An attacker who knows an email can trigger repeated messages within the six-per-fifteen-minute email-plus-IP limit, or distribute attempts across IPs.

How I would fix it: Add CAPTCHA or bot detection after suspicious activity, a per-email global limit, and email reputation monitoring with alerting.

How to say it: *"I have layered rate limits, but no CAPTCHA. I would add bot detection and per-email global limits if abuse appeared."*

Gap 11. The password policy is thin, and bcrypt truncates long inputs.

Weakness: Password validation is only a minimum of eight characters. There is no maximum, breach check, or complexity rule, and bcrypt truncates beyond 72 bytes.

Risk: Weak passwords are allowed, and very long passphrases can be silently truncated. bcryptjs cost 10 is also slower than native bcrypt.

How I would fix it: Add a maximum length, a breached-password check, and benchmark native bcrypt or Argon2id with a tuned cost.

How to say it: *"I hash with bcryptjs cost 10 and enforce a minimum length, but I do not claim a strong password policy. I would add breach checks and a max length, and benchmark Argon2id."*

Quick-Fire Concepts

14 rows

Use these one-line answers when a recruiter wants a fast definition before going deep.

Concept	My implementation	Why it matters
OTP format	6 digits, strings, padStart(6, '0')	Leading zeros survive; 1,000,000 values.
Generation	crypto.randomInt(0, 10**6)	CSPRNG, not Math.random.
TTL	10 minutes (OTP_TTL_MS)	Checked against expires_at in Postgres.
Hash	HMAC-SHA256(secret, userId:otp)	No plaintext OTP stored; user-bound.
Secret	OTP_HASH_SECRET, fallback JWT_SECRET	Dedicated key preferred; fallback is a gap.
Attempts	5 per OTP, database-enforced	Wrong attempts consume the code at five.
Resend cooldown	60 seconds	Database checks max(created_at) and returns retry_after.
Request limit	6 per 15 min per email plus IP	express-rate-limit, in-memory store.
Verify limit	20 per 15 min per email plus IP	HTTP layer, separate from database attempts.
Atomicity	RPC plus users row lock plus OTP row lock	Issue and verify are single transactions.
Active OTP	Unique partial index where consumed_at is null	One active code per user.
Email	Nodemailer SMTP, text plus HTML	Port 587 default; secure on 465.
Password	bcryptjs, cost 10	Automatic salt; compare only at login.
Verification gate	requireVerifiedEmail on POST /api/hackathons	Database check, not a JWT claim.

30-Second Project Pitch

1 pitch

Say this in about thirty seconds, then stop and let the interviewer choose a thread.

Signup posts to /api/auth/signup. I create the user with bcryptjs cost 10 and email_verified=false, generate a six-digit OTP with crypto.randomInt, HMAC-SHA256 it with OTP_HASH_SECRET, and call issue_email_verification_otp, which locks the user row, enforces a 60-second resend cooldown, consumes older codes, and inserts the new one. Nodemailer sends the code; the browser posts email plus OTP to /api/auth/verify-otp. verify_email_otp locks the user and OTP rows, checks the 10-minute expiry and five-attempt limit, consumes the code once, and sets email_verified. Unverified users can log in but are redirected to /verify-email, and submission is blocked server-side by requireVerifiedEmail. I have six node:test unit tests for generation, HMAC binding, expiry, cooldown, verify, and revoke.

SOURCE MAP

- backend/services/otpCrypto.js: OTP constants, generateOtp, hashOtp, resolveOtpSecret.
- backend/services/otpServiceCore.js: Issue, verify, and revoke orchestration.
- backend/repositories/otpRepo.js: Supabase RPC calls and cleanup.
- backend/sql/migrations/20261102_add_email_verification.sql: Table, indexes, RPCs, and grants.
- backend/routes/auth.js: Signup, verify-otp, resend-otp, and login behavior.
- backend/middleware/requireVerifiedEmail.js: Submission gate that rechecks the database.
- backend/services/notificationService.js: Nodemailer transport and OTP templates.
- frontend/src/pages/VerifyEmail.jsx: Six-digit input, resend cooldown, and redirect.
- backend/tests/otpService.test.js: Six node:test unit tests for pure OTP logic.